

# RESILIENCEOS

## Whole-Project Execution Map

20 Pre-Hackathon Days + 30-Hour Hackathon Build Plan

*AI-Powered Intelligent Hospital Digital Twin for Infrastructure Resilience*

**Execution objective**

Maximize the probability of a stable, judge-understandable, technically defensible demonstration by completing and validating the core loop before the event. This plan does not guarantee Rank 1; it is designed to reduce failure risk and increase demonstrable value.

| Planning horizon | Primary goal                                            | Exit condition                                        |
|------------------|---------------------------------------------------------|-------------------------------------------------------|
| Days 1–5         | Architecture, scope lock, data/configuration foundation | Team can run a minimal state transition locally       |
| Days 6–10        | Graph, cascade, service impact, scoring                 | Injected failure produces traceable downstream impact |
| Days 11–15       | What-if, dashboard, real-time, 3D synchronization       | One complete end-to-end demo loop works               |
| Days 16–20       | Validation, polish, rehearsal, freeze                   | Fallback-ready build and recorded evidence exist      |
| 30-hour event    | Integrate, stabilize, present                           | No critical P0 defect and rehearsed final pitch       |

# 1. Project Execution Doctrine

The reference specification defines ResilienceOS as a decision-support prototype for a small, representative hospital infrastructure model. The implementation must preserve the separation between infrastructure state and service-level impact, use synthetic state-driven telemetry, and require explicit human action for simulated interventions.

- Core operating loop: OBSERVE → UNDERSTAND → ESTIMATE → SIMULATE → COMPARE → EXPLAIN → HUMAN DECISION → RECOVER.
- Core reasoning engine: deterministic graph traversal, capacity/threshold rules, simulation, evaluation, and traceable explanation.
- ML boundary: optional anomaly detection or auxiliary signals; ML is not the causal engine and does not replace graph/rule reasoning.
- Digital-twin boundary: prototype digital twin of a simulated hospital infrastructure model, not a live hospital twin.
- Safety boundary: no patient data, no clinical decision-making, no real BMS/SCADA connection, and no autonomous actuation.
- All numbers must be labeled as configurable prototype assumptions or illustrative examples unless externally sourced.

## Terminology rule

Use “analyzes and simulates how failures may propagate through modeled dependencies.” Use “predict” only when referring to a clearly bounded estimate, such as time-to-threshold, and state the assumptions.

## 2. Scope Lock: What Must Be Built

### 2.1 P0 — Non-negotiable core

| Capability                  | Required behavior                                                                | Acceptance test                                                 |
|-----------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Configurable hospital model | Assets/services/edges/thresholds live in JSON/YAML/database config               | Change one capacity or dependency without editing engine logic  |
| State engine                | NORMAL / DEGRADED / AT RISK / FAILED for infrastructure; distinct service status | State transitions are deterministic and logged                  |
| Failure injection           | Operator selects origin, severity, duration                                      | Injection creates an event and visible origin status change     |
| Dependency graph            | Infrastructure and services connected with typed relationships                   | Trace path from failed asset to affected service                |
| Cascade engine              | Traversal plus dependency strength, redundancy, capacity and threshold checks    | Downstream impact is conditional, not automatic                 |
| Service impact              | ICU, OT, Emergency, Ward and non-critical service-level status/risk              | Service status differs from asset status where appropriate      |
| What-if simulation          | At least A/B/C stable; target A–F if feasible                                    | Same incident and horizon produce comparable strategy records   |
| Resilience Index            | Project-defined normalized 0–100 score with sub-scores                           | Score stays bounded and exposes C/A/B/T/U components            |
| Explainability              | 3–6 lines generated from graph path and simulation deltas                        | Every explanation sentence maps to an edge, threshold or output |
| Recovery                    | Simulated RECOVERING → NORMAL behavior                                           | Applied intervention visibly changes state over simulated time  |
| Human-in-loop               | Explicit operator action for applying a strategy                                 | No automatic strategy application                               |
| Demo fallback               | Seeded deterministic scenario and screenshots/video backup                       | Demo can continue if WebSocket/3D/ML fails                      |

### 2.2 P1 — Add after P0 is stable

- All six response strategies A–F with full comparison table.
- WebSocket streaming and synchronized 2D dashboard + 3D state.
- Incident timeline and time-to-threshold estimate.
- Water, HVAC and medical-gas secondary scenarios.
- Isolation Forest or robust statistical anomaly detector.
- Multiple simultaneous failure handling and richer recovery behavior.

### 2.3 P2 — Defer unless everything else is verified

- Real hospital/BMS/SCADA integration.
- Autonomous control or real actuator commands.

- Clinical or patient-outcome prediction.
- Multi-agent LLM architecture as a core dependency.
- Photorealistic 3D environment.
- Unvalidated optimization claims, certifications, or real-world performance claims.

### 3. Target Technical Architecture

| Layer               | Primary responsibility                                      | Suggested implementation           | Definition of done                                |
|---------------------|-------------------------------------------------------------|------------------------------------|---------------------------------------------------|
| Configuration       | Assets, services, edges, capacities, thresholds, priorities | JSON/YAML + Pydantic models        | Config loads and validates at startup             |
| Telemetry simulator | State-driven synthetic readings and events                  | Python state machine               | Normal, failure, backup and recovery trajectories |
| State engine        | Authoritative current status                                | Python domain module               | Single source of truth for node/service state     |
| Graph               | Typed dependencies and traversal paths                      | NetworkX MVP; Neo4j if stable      | Graph snapshot and path tracing work              |
| Cascade engine      | Conditional propagation                                     | BFS/queue traversal                | Returns ordered changes + paths + reasons         |
| Simulation          | Replay state forward under intervention                     | Copy-on-write state + shared rules | Same engine used for live and what-if             |
| Evaluation          | Comparable strategy metrics                                 | Deterministic formulas             | Same horizon/metrics for each strategy            |
| Explainability      | Traceable plain-language output                             | Templates from structured events   | No unsupported causal statements                  |
| API/realtime        | Actions, queries and updates                                | FastAPI + WebSockets               | Client can reconnect and resync                   |
| Dashboard           | State, incident, comparison, explanation                    | React/TypeScript                   | Judge understands screen hierarchy quickly        |
| 3D twin             | Visual state representation                                 | Three.js/R3F                       | Uses same state objects as dashboard              |
| Persistence         | Runs, telemetry, scenarios, logs                            | SQLite/PostgreSQL; Neo4j optional  | Demo run can be reconstructed                     |

### 4. Six-Person Team Operating Model

| Person                      | Primary ownership                                              | Secondary responsibility                 | Daily proof                       |
|-----------------------------|----------------------------------------------------------------|------------------------------------------|-----------------------------------|
| Member 1 — Backend Lead     | FastAPI, orchestration, state API, WebSockets                  | Integration and release branches         | Endpoint demo + logs              |
| Member 2 — Data/Graph       | Config schema, nodes, edges, graph queries                     | Synthetic telemetry schema               | Graph snapshot + traversal test   |
| Member 3 — Simulation Lead  | State machine, cascade, failure/recovery                       | Edge cases and deterministic seeds       | Scenario output + unit tests      |
| Member 4 — Evaluation/AI    | Risk, Resilience Index, strategies, anomaly option             | Metric validation and explanation inputs | Score table + formula test        |
| Member 5 — Frontend/Product | Dashboard UX, incident timeline, comparison, explainability UI | Judge-facing information hierarchy       | Screen recording + usability pass |
| Member 6 — 3D/Demo/QA       | 3D twin, visual states, demo orchestration                     | Fallback, rehearsal, evidence pack       | 3D sync test + backup demo        |

#### Branching rule

Use one protected main branch, short feature branches, daily integration, and a release tag at the end of every major checkpoint. No unreviewed feature merges after the Day 17 freeze candidate.

### 5. Twenty-Day Pre-Hackathon Roadmap

The 20 days are divided into four five-day sprints. Each sprint ends with a demonstrable checkpoint. If a sprint slips, remove P1 features before sacrificing P0 stability.

| Day   | Focus                     | Execution tasks                                                                                  | Exit deliverable                                        |
|-------|---------------------------|--------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Day 1 | Scope & architecture lock | Confirm P0/P1/P2, hero scenario, naming, repo structure, API contracts                           | Approved scope sheet, architecture diagram, issue board |
| Day 2 | Configuration foundation  | Define asset/service/edge schemas, capacities, thresholds, priorities, recovery behavior         | Validated config file + sample hospital model           |
| Day 3 | State engine              | Implement node/service states, transitions, event model, deterministic clock                     | Unit tests for all state transitions                    |
| Day 4 | Synthetic telemetry       | Build state-driven simulator for normal, grid failure, generator start, UPS/battery and recovery | Repeatable telemetry stream with seed                   |

| Day    | Focus                       | Execution tasks                                                                             | Exit deliverable                                   |
|--------|-----------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------|
| Day 5  | Sprint 1 checkpoint         | Integrate config → simulator → state engine; record baseline run                            | Minimal system runs without UI                     |
| Day 6  | Graph implementation        | Create infrastructure/service nodes and typed relationships                                 | Graph loads from configuration                     |
| Day 7  | Traversal & paths           | BFS/queue traversal, capped depth, path capture, multi-origin seed support                  | Traceable dependency path output                   |
| Day 8  | Capacity/threshold logic    | Dependency strength, redundancy, threshold crossing, backup competition                     | Conditional propagation tests                      |
| Day 9  | Service impact              | Compute ICU/OT/Emergency/Ward/non-critical impact separately from asset status              | Service-risk scenario report                       |
| Day 10 | Sprint 2 checkpoint         | Run main power failure from origin to service impact and recovery                           | CLI or notebook demo + test evidence               |
| Day 11 | What-if engine              | Implement shared simulation loop and intervention overrides                                 | Strategy A/B/C working                             |
| Day 12 | Response strategies         | Add D/E/F only after A/B/C are stable; define exact effects                                 | Comparable records with same horizon               |
| Day 13 | Resilience Index            | Normalize continuity, availability, backup margin, recovery and resource terms; test bounds | Formula document + automated boundary tests        |
| Day 14 | Explainability              | Generate causal chain and strategy rationale from structured evidence                       | Why-risk and why-strategy outputs                  |
| Day 15 | Sprint 3 checkpoint         | Complete backend flow: inject → cascade → what-if → score → explain → recover               | End-to-end API or script demo                      |
| Day 16 | Dashboard core              | Current state, active incident, affected assets/services, response panel                    | Judge-readable 2D dashboard                        |
| Day 17 | Realtime integration        | WebSocket events, reconnect/resync, incident timeline                                       | Live state updates without page refresh            |
| Day 18 | 3D synchronization          | Medium-detail hospital/utility scene, status colors/animations, shared state stream         | 2D and 3D show same state                          |
| Day 19 | Validation & demo hardening | Failure cases, missing telemetry, duplicate events, reset, performance and logs             | P0 test report + fallback package                  |
| Day 20 | Freeze & rehearsal          | Record demo, finalize pitch, Q&A, screenshots, backup video, release tag                    | Release candidate v1.0-demo and no open P0 defects |

## 6. Daily Execution Cadence During the 20 Days

| Time block | Activity                              | Output                                   |
|------------|---------------------------------------|------------------------------------------|
| 15 min     | Stand-up: yesterday / today / blocker | Single blocker owner and deadline        |
| 2–4 hrs    | Focused implementation                | Commit linked to an issue                |
| 30 min     | Integration window                    | Main branch stays runnable               |
| 30 min     | Test + screenshot evidence            | Proof of behavior, not verbal status     |
| 15 min     | Risk review                           | Move features between P0/P1/P2 if needed |
| End of day | Demo from clean start                 | Short recording or checklist sign-off    |

### Definition of progress

A feature is not “done” because code exists. It is done only when it runs from a clean start, has a visible output, has at least one test or reproducible scenario, and is labeled honestly as IMPLEMENTED.

## 7. Mandatory Checkpoints and Go/No-Go Gates

| Gate                    | When   | Must be true                                             | If failed                                          |
|-------------------------|--------|----------------------------------------------------------|----------------------------------------------------|
| G1 — Architecture       | Day 1  | Scope, contracts, ownership and hero scenario approved   | Remove nonessential modules; freeze interfaces     |
| G2 — Engine             | Day 5  | Config + simulator + state engine run deterministically  | Drop ML/3D work temporarily                        |
| G3 — Cascade            | Day 10 | Failure reaches service-level impact with traceable path | Use NetworkX and simpler rules; no Neo4j migration |
| G4 — Decision loop      | Day 15 | What-if, score, explain and recovery work end-to-end     | Limit strategies to A/B/C and simplify UI          |
| G5 — Visual integration | Day 18 | Dashboard and 3D share one state stream                  | Use dashboard as primary and 3D as optional panel  |
| G6 — Release            | Day 20 | Recorded demo, fallback and no P0 defect                 | Freeze features; repair only reliability defects   |

## 8. Resilience Index Implementation Contract

The Resilience Index is project-defined and must not be presented as a standardized hospital-safety score. Every component must be normalized to [0,1], and the final score must be bounded to [0,100].

| Symbol | Meaning                     | Implementation requirement                                            |
|--------|-----------------------------|-----------------------------------------------------------------------|
| C      | Critical-service continuity | Criticality-weighted average of 1 – service risk for ICU/OT/Emergency |
| A      | Infrastructure availability | Fraction of assets in NORMAL or DEGRADED, with explicit denominator   |

| Symbol | Meaning               | Implementation requirement                                                   |
|--------|-----------------------|------------------------------------------------------------------------------|
| B      | Backup margin         | Normalized generator headroom and battery reserve                            |
| T_norm | Recovery-time penalty | Recovery duration divided by configured maximum/reference, capped to 1       |
| U_norm | Resource-use penalty  | Load shed/resource consumption relative to configured reference, capped to 1 |
| R      | Composite score       | 100 × weighted combination, then clamp to [0,100] if required                |

### Formula safeguard

Do not leave the weighted expression ambiguous. Define whether weights are normalized before or after penalties, define all denominators and zero-division cases, and display C/A/B/T/U beside the composite score.

## 9. What-If Strategy Build Order

| Strategy                            | Implementation meaning                                                        | Build priority |
|-------------------------------------|-------------------------------------------------------------------------------|----------------|
| A — Do Nothing                      | No intervention; comparison baseline                                          | P0             |
| B — Activate Backup                 | Start/connect generator with configured ramp-up                               | P0             |
| C — Generator + Reduce Non-Critical | Activate backup and shed configured non-critical load                         | P0             |
| D — Redistribute Loads              | Shift allocation/priority between buses/services                              | P1             |
| E — Prioritize Critical Services    | Allocate available backup capacity to ICU/OT/Emergency before remaining loads | P1             |
| F — Combined                        | Generator + shedding + redistribution + critical-service prioritization       | P1             |

Use the phrase “best-performing simulated response under configured prototype objectives.” Do not call a strategy universally safest, optimal, guaranteed, or validated.

## 10. Recommended Hero Demonstration

| Sequence       | Approx. time | What the judge sees                                              | Narrative anchor                                                |
|----------------|--------------|------------------------------------------------------------------|-----------------------------------------------------------------|
| Problem        | 0:00–0:20    | Hospital infrastructure dependencies and service criticality     | A local failure can propagate across dependent systems          |
| Normal state   | 0:20–0:35    | Stable index, green/normal assets, service overview              | We begin from a known baseline                                  |
| Inject failure | 0:35–0:55    | Transformer/grid failure event                                   | The operator deliberately starts a simulated incident           |
| Cascade        | 0:55–1:25    | Bus stress, generator/UPS response, battery margin, service risk | The graph explains what is affected next                        |
| What-if        | 1:25–2:05    | A/B/C or A–F comparison with common metrics                      | We test responses before applying one                           |
| Explain        | 2:05–2:25    | Why-risk and why-strategy causal chain                           | Every statement is grounded in graph/simulation evidence        |
| Recovery       | 2:25–2:50    | Operator applies selected simulated strategy; states recover     | Human decision remains in control                               |
| Close          | 2:50–3:00    | Scope, limitation, future integration                            | Prototype demonstrates decision support, not autonomous control |

### Fallback demo order

If any live component fails: use the seeded backend scenario, then the recorded screen capture, then static screenshots. Never improvise unverified numbers or claim that an unimplemented feature is live.

## 11. Thirty-Hour Hackathon Execution Schedule

The event schedule assumes the team arrives with the pre-hackathon release candidate, seeded configuration, working demo script and a tested fallback. The hackathon is for integration, stabilization, evidence and presentation—not for inventing the architecture from scratch.

| Time    | Phase                              | Tasks                                                                     | Exit condition                                                     |
|---------|------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------|
| H0–H1   | Setup and environment verification | Clone, install, env variables, database/graph availability, branch cut    | All six machines can run the project or use shared dev environment |
| H1–H3   | Baseline integration               | Run normal state, verify config load, confirm API and dashboard boot      | Baseline screenshot + logs                                         |
| H3–H6   | Hero scenario integration          | Inject failure, observe cascade, verify service impact and event timeline | Main incident works end-to-end                                     |
| H6–H8   | What-if and scoring                | Run A/B/C; verify comparable metrics and bounded index                    | Comparison table + score tests                                     |
| H8–H10  | Explainability and recovery        | Why-risk output, apply simulated intervention, recovery sequence          | Full decision loop                                                 |
| H10–H12 | 3D and UI synchronization          | Connect 3D state, visual states, camera framing, labels                   | 2D/3D consistency check                                            |
| H12–H14 | First full rehearsal               | Run timed 3-minute demo; list every friction point                        | Bug list ranked P0/P1/P2                                           |

| Time    | Phase                                 | Tasks                                                                           | Exit condition                                   |
|---------|---------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------|
| H14–H17 | Reliability sprint                    | Fix crashes, reset issues, stale WebSocket state, bad labels, score bugs        | No known P0 defect in hero flow                  |
| H17–H19 | Secondary scenario                    | Add or verify generator/UPS failure; water/HVAC only if stable                  | One additional scenario or documented limitation |
| H19–H21 | Evidence and documentation            | Screenshots, test output, architecture slide, limitations, source/status labels | Evidence folder complete                         |
| H21–H23 | Pitch and Q&A                         | 30-second, 3-minute and 5-minute versions; assign speaking roles                | Each member answers their module questions       |
| H23–H25 | Second rehearsal                      | Run with one person acting as judge; no developer narration shortcuts           | Judge can follow the UI                          |
| H25–H27 | Freeze candidate                      | Tag release, backup repo/archive, record clean demo, export screenshots         | Rollback point exists                            |
| H27–H29 | Final polish only                     | Fix only high-confidence reliability or clarity defects                         | No risky architectural changes                   |
| H29–H30 | Submission and presentation readiness | Verify files, links, video, build command, team roles, final reset              | Ready-to-present package                         |

## 12. Verification and Testing Plan

| Test category | Minimum tests                                                                | Evidence                      |
|---------------|------------------------------------------------------------------------------|-------------------------------|
| Unit          | State transitions, threshold checks, normalization, index bounds             | Automated test output         |
| Graph         | Path traversal, capped depth, missing edge, duplicate node, multiple origins | Traversal snapshots           |
| Simulation    | Same seed produces same output; strategy does not mutate baseline            | Repeat-run comparison         |
| Failure       | Grid/transformer, generator, UPS/battery, water/HVAC/gas where implemented   | Scenario result files         |
| API           | Injection, what-if, explanation, apply strategy, reset                       | Endpoint checks / logs        |
| Realtime      | Reconnect, resync, stale client, rapid state changes                         | Screen recording or checklist |
| UI            | Readable labels, no unexplained color-only states, clear active incident     | Judge usability review        |
| Fallback      | Demo runs without ML, without 3D, and from recorded evidence                 | Fallback rehearsal sign-off   |

## 13. Risk Register and Mitigation

| Risk                                 | Probability/impact | Mitigation                                                | Decision trigger                             |
|--------------------------------------|--------------------|-----------------------------------------------------------|----------------------------------------------|
| 3D consumes too much time            | High / High        | Use medium-detail scene; dashboard remains primary        | If 3D blocks P0, freeze 3D                   |
| Neo4j setup/integration delay        | Medium / High      | Use NetworkX MVP with graph abstraction                   | If not stable by Day 8, stay on NetworkX     |
| ML produces weak or confusing output | Medium / Medium    | Use statistical fallback; keep ML auxiliary               | If no testable value by Day 15, defer        |
| WebSocket instability                | Medium / High      | REST snapshot + reconnect/resync + recorded demo          | If unstable by Day 17, use snapshot fallback |
| Index formula bug                    | Medium / High      | Normalize components, boundary tests, show sub-scores     | No release without bounded-score tests       |
| Overclaiming realism                 | Medium / High      | Use labels: synthetic, illustrative, prototype assumption | Review every slide and UI claim              |
| Scope creep                          | High / High        | P0/P1/P2 triage every day                                 | Any new feature requires removing another    |
| Demo reset failure                   | Medium / High      | Seeded reset endpoint and clean-start script              | Rehearse 5 consecutive clean runs            |
| Team integration conflict            | Medium / High      | Daily main-branch integration and API contracts           | Stop parallel incompatible changes           |

## 14. Evidence Pack to Prepare

- Architecture diagram showing simulator, state engine, graph, cascade, what-if, evaluation, explanation, API, dashboard and 3D.
- Configuration file excerpt proving that topology and thresholds are data-driven.
- Test output for state transitions, cascade propagation, index bounds and deterministic replay.
- Before/after screenshots for normal state, injected failure, cascade and recovery.
- What-if comparison screenshot with identical metrics and time horizon.
- Traceable explanation screenshot with the actual dependency path and threshold reason.
- A short clean demo recording and a separate backup recording.
- Limitations sheet: synthetic data, prototype assumptions, no live hospital integration, no clinical claims, no autonomous control.
- Build/run instructions that a judge or mentor can follow without team-specific hidden steps.

## 15. Judge Q&A Preparation

| Question                     | Answer structure                                                                                                               |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Is this a real digital twin? | It is a prototype digital twin of a simulated hospital infrastructure model; no live physical hospital telemetry is connected. |
| Why use a graph?             | The core question is variable-depth dependency traversal: what does a failure reach and through which                          |

| Question                              | Answer structure                                                                                                                                            |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                       | path?                                                                                                                                                       |
| Why not only ML?                      | Causal propagation and threshold behavior must remain traceable; ML is auxiliary for anomaly signals or uncertainty.                                        |
| Where did the data come from?         | State-driven synthetic telemetry generated from configured scenarios; no real hospital or patient data is used.                                             |
| Does it control hospital equipment?   | No. It is human-in-the-loop decision support and only applies interventions inside a simulation sandbox.                                                    |
| Is the Resilience Index a standard?   | No. It is a project-defined normalized prototype metric with configurable weights and visible sub-scores.                                                   |
| Can it guarantee service continuity?  | No. It demonstrates modeled behavior under configured assumptions; real deployment would require facility-specific engineering validation.                  |
| What is novel?                        | The prototype combines dependency-aware cascade analysis, intervention comparison, service-level impact and traceable explanation in one operator workflow. |
| What happens if telemetry is missing? | The system should flag missing data, avoid silently inventing certainty, and fall back to a known state or scenario-level limitation.                       |

## 16. Status and Claim Governance

| Label                  | Use it when                                           | Example                                   |
|------------------------|-------------------------------------------------------|-------------------------------------------|
| [IMPLEMENTED]          | Built, runnable and verified in the current prototype | Failure injection passes integration test |
| [PROPOSED / DESIGN]    | Specified but not confirmed working end-to-end        | Neo4j deployment plan                     |
| [FUTURE EXTENSION]     | Explicitly outside the event scope                    | Live BMS/SCADA integration                |
| [PROTOTYPE ASSUMPTION] | Configurable simplification or chosen parameter       | Illustrative generator capacity           |
| [ILLUSTRATIVE EXAMPLE] | Worked number used only to explain behavior           | Example index after failure               |

Before submission, audit every slide, UI label, README statement and verbal claim. Remove words such as guaranteed, optimal, safest, clinically validated, real-time hospital deployment, or accurate prediction unless the team has direct evidence supporting the exact claim.

## 17. Final Release Checklist

- P0 hero scenario runs from clean start at least five consecutive times.
- Failure injection, cascade, service impact, what-if, index, explanation and recovery all work or are honestly labeled.
- Index remains in [0,100] and displays sub-scores.
- A/B/C strategies are stable; D/E/F are included only if tested.
- Dashboard and 3D do not disagree about state; if they do, dashboard is the source of truth.
- WebSocket failure does not corrupt backend state; REST snapshot or recording fallback exists.
- All demo numbers are labeled synthetic, illustrative or prototype assumptions.
- No patient data, clinical recommendation, autonomous control or compliance claim is presented.
- Team members know their ownership, one-sentence explanation, and likely judge questions.
- Repository is tagged, backed up, and runnable using documented commands.
- Final pitch has 30-second, 3-minute and 5-minute versions.
- Backup video, screenshots, architecture slide and limitations slide are available offline.

## 18. Final Direction

### The winning execution mindset

Do not attempt to prove that ResilienceOS solves every hospital resilience problem. Prove, clearly and reliably, that your team can model dependencies, trace a failure, simulate interventions, compare outcomes, explain the result, and keep the human operator in control. A smaller system that runs repeatedly and can defend every assumption is stronger than a larger system that only exists in documentation.

Recommended priority order: reliability → causal traceability → judge comprehension → comparable what-if outcomes → visual polish → optional ML breadth. Protect the core loop at every checkpoint.

**Source basis:** Prepared from the attached ResilienceOS Master Technical & Product Documentation (Draft — Hackathon Prototype Specification), preserving its stated scope, architecture, safety boundaries, synthetic-data framing, graph/cascade model, what-if strategies, Resilience Index concept, and human-in-the-loop design.